Lecture 1: Intro to Computation, Basic Julia, and Version Control

Computational Bootcamp

John P. Ryan

Summer 2026

University of Wisconsin–Madison

Today's goals

1. Understand what computation is and how it relates to what we do in economics
2. Learn the basics of Julia
3. Set up Git and GitHub for version control
4. Do some coding

Computation in Economics

What is computation in Economics?

- Any time you do math on a computer, you are computing!
- Some computation you might have already done before:
  - Create new variables in a dataset
  - Analyze data with built-in functions
- Other kinds of computation:
  - Optimize a function (i.e. find a maximum or minimum)
  - Approximate an equation without a closed-form solution
  - Solve dynamic systems of equations

Why do we need computation in Econ?

1. Enlarges the set of questions we can tackle
2. Improves the quality of our answers

- If you don't do computation, you can use theory and proofs
  - Even then, computation can guide and motivate your theory
- If you only know how to use Stata, you can work with data
  - Working with data requires some (or a lot of) computation
  - What if you want to build a structural model?
  - What if the function or package you need doesn't exist?
- In both cases, your research will be limited without strong computational skills

What does this mean for you?

- Computation will be an important part of your job:
  - In your own work
  - Dealing with co-authors and colleagues
  - Reading and evaluating the work of others
- So, you should learn how to do computation well now!
- If you are a rising second year, your goal is to:
  1. Figure out what questions you are interested in
  2. **Acquire the tools (+ data) you need to answer those questions**
- This process will likely continue after the second year

How do you do computation well?

Your code should be:

1. **Fast**
   - How long does it take to *write* your code?
   - When will my code finish *running*?
2. **Readable**
   - Can other people understand your code?
   - Can the future version of you understand your code?
3. **Reproducible**
   - Does your code give the correct answers?
   - How easy would it be to change/extend your code?

How do you do computation well?

- These things are all related:
  1. If your code is not readable: it is harder to find mistakes, so it is not reproducible
  2. If your code is not reproducible: it is difficult to make iterative progress, so it is not fast
  3. If your code takes 10 years to finish running (not fast), then there's no need to read it or reproduce it (useless)
- Luckily, there is good software available to make this easier:
  - Julia and Visual Studio Code for programming
  - Git and GitHub for version control
  - AI coding assistants, *used carefully* (more on this later)

Why Julia?

Why Julia?

- Pros:
  - Faster runtime than basic Matlab, Python, R, Stata
  - Easier to learn and work in than C++ and Fortran
  - More readable than optimized Matlab or Python code
  - Open source, with a (mostly) mature ecosystem for scientific computing
- Cons:
  - C++ and Fortran can have faster runtimes (sometimes)
  - You might already know other languages (switching costs)
  - Smaller package ecosystem than Python/R (especially for data work)
  - Error messages can be a bit puzzling initially - lots of type errors

My advice

1. You should learn either Stata, R, or Python well
   - For data cleaning and basic analysis
   - Use the large library of packages and legacy code
   - Stick with what you already know best
2. You should learn Julia well
   - For high-performance computing needs
   - This camp and Econ 880 (Computational Economics) will help with this
3. Learn and use other coding languages as needed
   - Each new language is easier to learn than the last
   - Example: Python for scraping/data work, R for data, Julia for model solving/estimation

Getting set up

- Install Julia with juliaup, the official version manager:
  - `https://julialang.org/downloads/` — one command installs and manages versions
  - As of this summer, the current stable release is Julia 1.12
- Install Visual Studio Code and the Julia extension
  - REPL integration, plots pane, debugger, profiler
- Package management happens in the REPL with `]` (Pkg mode):

```
julia> ]                    # enter Pkg mode
(@v1.12) pkg> add Plots     # install a package
(@v1.12) pkg> status        # list installed packages
```

- Later we will use per-project environments (`Project.toml`) so your code runs on any machine

Version Control with Git

Why version control?

- Does this folder look familiar?
  - `model_final.jl`, `model_final_v2.jl`, `model_final_v2_REAL.jl`, ...
- Git tracks the history of your code so that you can:
  - Roll back to any previous version
  - See exactly what changed, when, and why
  - Work on risky changes in a *branch* without breaking working code
  - Collaborate with co-authors without emailing zip files or paying for Dropbox
- GitHub hosts your Git repositories online:
  - Off-site backup of your research code
  - The standard way to share code (replication packages!)
  - You will submit homework via Github

The core Git workflow

You will use five commands for 95% of your work:

```
git clone <url>          # copy a repository to your computer, or use ‘git init’ to start from scratch
git status               # what has changed?
git add file.jl          # stage changes for the next snapshot
git commit -m "Add VFI solver"   # take a snapshot
git push                 # upload snapshots to GitHub
git pull                 # download others’ snapshots
```

- Commit early and often — small commits with informative messages
- VS Code has this all built in (Source Control panel), so you rarely need the command line

Version control tips

- Commit code, not output: use a `.gitignore` file to exclude large data files, results, and junk (e.g. `.DS_Store`)
  - GitHub blocks files over 100MB — keep big data out of the repository
- Write commit messages your future self can search: "Fix off-by-one in capital grid" beats "stuff"
- One repository per project, with a sensible folder structure (`code/`, `data/`, `output/`)
- Sign up for the GitHub Student Developer Pack (free private repositories and other goodies)
- Resources:
  - `https://swcarpentry.github.io/git-novice/`
  - `https://happygitwithr.com` (R-focused but excellent on concepts)

Let's go!

It's coding time!